

Schema-Augmented Retrieval for Text-to-SQL: Natural Language Access to the Chicago Crime Database

Vignesh, Rupesh, Alexander, Zhonping, and Fayeq

Abstract

As data-driven decision-making becomes increasingly critical across organizations, non-technical stakeholders often cannot write SQL queries to extract insights from databases. To address this Text-to-SQL natural language processing (NLP) problem, this project proposes a SQL chatbot built with prompt engineering. The system is designed as an AI agent using LangChain or LangGraph to manage interactions. The agent translates natural language into SQL queries, executes them against the database, and summarizes the resulting data back into natural language. The system’s query generation accuracy and robustness are evaluated using the Chicago Crime dataset, a large-scale structured dataset containing records of criminal incidents with attributes such as location, time, and offense type. This project aims to expand data access by removing the requirement for SQL knowledge.

1 Introduction

Relational databases are critical for operational intelligence, but their data remain largely inaccessible to non-technical stakeholders who lack SQL proficiency. This barrier forces business users to rely on engineering teams, creating operational bottlenecks and delaying actionable insights.

Although LLMs have introduced Text-to-SQL capabilities, current systems struggle in real-world settings. Low-parameter or locally hosted models frequently hallucinate schemas, generate invalid queries, and lack intrinsic self-correction capabilities. Recent surveys confirm that these limitations persist across the LLM-era Text-to-SQL landscape [1, 2, 3]. Furthermore, many systems stop at query generation, leaving users to interpret raw tables.

This project addresses these limitations by developing an autonomous Text-to-SQL agent that uses an iterative LangGraph loop to compensate for the reasoning limitations of accessible models, producing accurate SQL and synthesizing results into conversational language.

1.1 Literature Survey

Text-to-SQL research has advanced through schema-aware encoding and in-context learning, benchmarked on datasets such as Spider [4] and models like RAT-SQL [5]. Decomposed prompting frameworks — DAIL-SQL [6] and DIN-SQL [7] — leverage chain-of-thought reasoning [8] and dynamic example selection to significantly improve execution accuracy, but depend on large proprietary LLMs such as GPT-4 [9] that present cost and accessibility barriers. These static pipelines fail on resource-constrained models like SQLCoder [10] that cannot self-correct syntax errors.

Multi-agent frameworks such as MAC-SQL [11] address static generation via ReAct-style [12] iterative observe-execute loops for autonomous error recovery, but are computationally heavy and return raw data tables without user-facing summaries. This project adapts DIN-SQL’s decomposed prompting and MAC-SQL’s iterative correction for low-parameter models using LangGraph [13], evaluated under the real-world complexity requirements established by BIRD [14, 15]. A final natural language synthesis step fully closes the operational gap for non-technical users. The key contributions are: (1) a novel end-to-end agentic architecture with a self-correction loop that narrates results in plain English; and (2) a systematic evaluation of prompt engineering techniques quantifying performance deltas in Execution Accuracy (EX) and Valid SQL Rate (VSR) across open-source models.

2 Materials and Methods

2.1 Dataset and Data Processing

The **Chicago Crime Dataset** ([Chicago Data Portal](#)) comprises approximately 8.5 million crime records spanning 2001–present, with 22 attributes covering categorical (crime type, location description, beat/ward/district), temporal (date, year), geospatial (latitude/longitude, coordinates), and metadata (case number, arrest, domestic) fields. Data is ingested weekly via an Airflow ETL pipeline into a Dockerized PostgreSQL v14+ instance. Preprocessing standardized all column names to `UPPERCASE_SNAKE` format to eliminate LLM double-quoting overhead, cast string timestamps to PostgreSQL `TIMESTAMP`, and cast coordinate fields to `FLOAT8`. Null values were handled per column group as shown in Table 1. Weekly Airflow schema validation confirms structural consistency before agent interactions.

Column Group	Specific Columns	Handling Strategy	Rationale
Categorical	CRIME_TYPE, DESCRIPTION, LOCATION_DESCRIPTION	Changed as OTHER OFFENSE	Permits downstream COALESCE operations in SQL
Geospatial	LATITUDE, LONGITUDE, X_COORDINATE, Y_COORDINATE	Replaced with the average of BEAT if not found a match with BLOCK	Allows spatial queries to filter valid records
Temporal	DATE, UPDATED_ON, YEAR	Updated the DATE column with frequent date for the YEAR	Malformed dates trigger query failures; ensures temporal consistency
Logical	ARREST, DOMESTIC	Marked as “Unknown”	Ensures “Unknown” status distinction (not executed vs. unknown arrest status)

Table 1: Null value handling strategy by column group.

2.2 Problem Formulation

The task is **Text-to-SQL translation**: given a natural language question from a non-technical user, produce a syntactically valid PostgreSQL `SELECT` statement, execute it, and return a plain-English summary structured as `{sql_query: str, results: List[Tuple], answer: str}`. The system enforces read-only access, rejecting `INSERT/UPDATE/DELETE/DROP/ALTER` statements, and restricts scope to the Chicago Crime database. Supported query patterns include `JOIN`, `GROUP BY`, `HAVING`, `ORDER BY`, `LIMIT`, and subqueries. The schema is assumed stable between weekly validations, and a 7-day data refresh lag applies to real-time queries.

2.3 Model / System Architecture

2.3.1 System Workflow

Weekly data ingestion via Airflow populates a Dockerized PostgreSQL database. The LangGraph AI Agent receives user questions, queries the database, and returns natural language answers. Figure 1 shows the overall system architecture; Figure 2 details the internal agent graph.

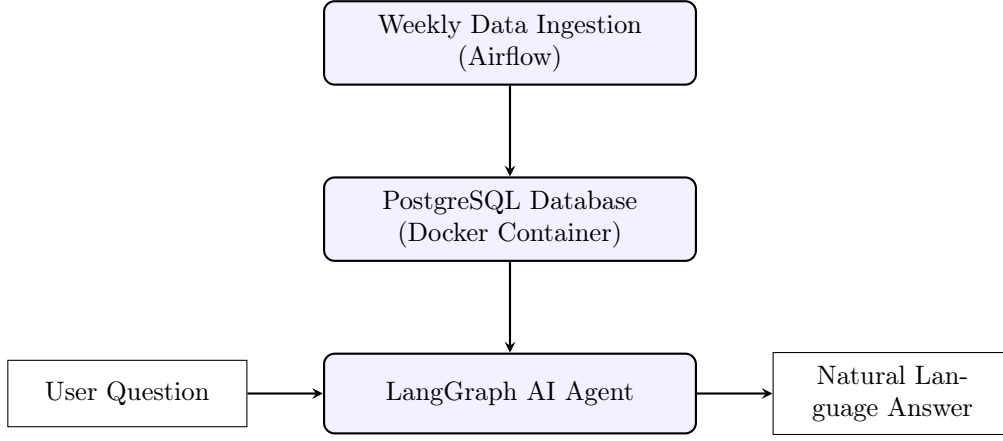


Figure 1: Overall system architecture: data pipeline and agent interaction flow.

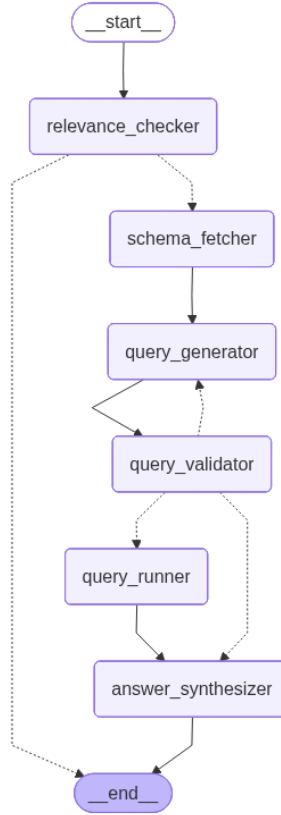


Figure 2: LangGraph AI Agent internal StateGraph architecture.

2.3.2 Six-Node StateGraph Architecture

The agent is implemented as a LangGraph `StateGraph` whose shared state carries `question`, `sql_query`, `schema`, `results`, `answer`, `error_log`, `retry_count`, and `is_relevant`. Six nodes execute in sequence, described in Table 2.

Conditional edges: The Relevance Checker routes to the Schema Fetcher (relevant) or END (irrelevant). The Query Validator loops back to the Query Generator on failure (max 3 retries), then falls through to the Answer Synthesizer with a user-friendly error message. FAISS [17] supports future vector-based retrieval extensions via efficient inner-product search over dense embeddings.

Node	Purpose	Key Implementation Detail
1. Relevance Checker	Filters non-database questions	Regex + keyword fast-path (<5 ms); outputs <code>is_relevant</code> flag; routes to END or Schema Fetcher
2. Schema Fetcher	Retrieves live DB schema	<code>SQLDatabase.get_table_info()</code> ; 50–200 ms database round-trip
3. Query Generator	LLM generates SQL	Strict system prompt: double-quote all identifiers, no DML, return <code>INVALID_REQUEST</code> if unanswerable; SBERT [16] few-shot retrieval; temp=0, max_tokens=500
4. Query Validator	Dry-runs SQL syntax	<code>EXPLAIN</code> + DML regex check; increments retry counter (max 3 attempts)
5. Query Runner	Executes validated SQL	<code>db.run(fetch="all")</code> ; caps result set to 10 rows for synthesis
6. Answer Synthesizer	Narrates results in plain English	LLM summarizes JSON result; returns pre-set message for rejected queries

Table 2: Six-node StateGraph: node functions and implementation details.

2.3.3 Tools and Frameworks

Component	Technology	Rationale
LLM	Groq API (openai/gpt-oss-120b)	Fast inference (<1s/query); cost-effective; supports long context
Agentic Framework	LangGraph	Explicit state management; transparent execution flow; conditional routing
Database Driver	LangChain SQLDatabase + SQLAlchemy	Abstraction over PostgreSQL dialect; built-in schema introspection
Database	PostgreSQL v14+	Robust; supports complex queries; production-ready
Containerization	Docker	Environment reproducibility; isolation; scaling
Data Pipeline	Apache Airflow	Scheduled weekly ingestion; automatic error handling; audit trail
Language	Python 3.11+	Mature ML ecosystem; LangChain/LangGraph native support

Table 3: Tools and frameworks used in the proposed system.

2.4 Training Strategy

The agent uses **iterative prompt engineering** as its sole optimization mechanism; no model fine-tuning is performed. Development proceeded through four phases: (1) a minimal baseline prompt that suffered high syntax errors and DML leakage; (2) explicit PostgreSQL constraints (double-quote all identifiers, block DML) combined with post-generation regex validation; (3) embedding actual Chicago Crime column names and example queries to reduce hallucinations and improve JOIN and LIMIT accuracy; and (4) separating relevance checking from query generation and adding EXPLAIN-based retry validation with refined user-facing error messages. Key hyperparameters are summarized in Table 4.

2.5 Evaluation Framework

Performance is assessed on a hand-crafted benchmark of 100 natural language questions paired with ground-truth SQL, stratified as 30 **Easy** (single-table aggregations such as COUNT/SUM), 40 **Medium**

Parameter	Value	Rationale
Temperature	0	Deterministic SQL; reproducibility; no random hallucinations
Max Tokens	500	Sufficient for complex multi-table queries; prevents token waste
Retry Limit	3	Balances accuracy (multiple attempts) vs. latency (not excessive)
Relevance Threshold	2+ action verbs + data context	Prevents false positives (generic questions)
Result Limit	3 rows (default)	Concise answers; token efficiency; user-friendly output
LLM Model	openai/gpt-oss-120b	Fast inference; good quality; supports 32k context

Table 4: Key hyperparameters used in the system.

(JOINS, GROUP BY with HAVING, temporal range analysis), and 30 **Hard** (window functions, CTEs, complex date arithmetic). Three metrics are used: **Execution Accuracy (EX)** — percentage of queries returning a result set identical to ground truth (target: 85%); **Valid SQL Rate (VSR)** — percentage of syntactically valid, executable queries (target: 95%); and **Synthesis Quality (SQ)** — mean Likert score (1–5) via hybrid manual and LLM-as-a-Judge annotation (target: ≥ 4.0). Baselines include a rule-based lexical matcher (expected EX $\sim 40\%$) and a zero-shot LLM without database-specific rules (expected EX $\sim 60\%$). Evaluation proceeds through four stages: query execution, result comparison against a PostgreSQL replica with a 5-second timeout, metric grading, and error categorization by failure type and difficulty tier.

2.6 Implementation Details

The system is built in Python 3.11+ using LangChain (v0.1+), LangGraph (v0.2+), Groq SDK (v0.9+), SQLAlchemy (v2.0+), psycopg2-binary (v2.9+), and Apache Airflow (v2.8+). The database runs as a `postgres:14-alpine` Docker container with 15 GB storage; LLM inference is handled server-side by Groq (30 req/min free-tier limit, CPU-based client).

Assumptions and Limitations. The schema is assumed stable between weekly Airflow validations; a 7-day data refresh lag applies to real-time queries; and the system operates as a single-user, single-connection session. Key limitations: scope is restricted to the Chicago Crime database; window functions and CTEs have limited support; and there is no built-in multi-user session management.

3 Results

3.1 Overall Performance

Table 5 compares the Baseline (V1) and Improved (V2) systems on the 100-question benchmark across three metrics: **VSR** (proportion of syntactically valid, executable queries), **EX** (result-set identity with ground truth using a hybrid relational equivalence metric; strict JSON-match EX: V1 12%, V2 19%), and **SQ** (mean Likert 1–5 faithfulness score).

V2 outperforms V1 on all three metrics: VSR +6 pp (87%→93%), EX +13 pp (47%→60%), and SQ +0.43 (3.91→4.34). V2 SQ of 4.34 exceeds the 4.0 target while V1 fell marginally short at 3.91.

3.2 Per-Tier Trends

Easy tier. V1 achieved SQ of 4.53, with failures caused by the `LIMIT 10` system-prompt instruction truncating multi-row enumerations (e.g., “List all distinct crime types” returned 10 of 31 types). V2 eliminates this entirely, achieving VSR = 100% and SQ = 5.00 across all 30 questions.

Medium tier. V1 VSR dropped to 78% due to relevance-checker false rejections on phrases like “which blocks”. V2’s expanded keyword list raised VSR to 92% and SQ from 3.62 to 4.20. Three

Table 5: System performance comparison across complexity tiers. EX is measured with a hybrid relational equivalence metric: column-name-agnostic value-set comparison, escalating to an LLM judge for borderline cases (strict JSON-match EX: V1 12%, V2 19%).

Tier (n)	Baseline (V1)			Improved (V2)		
	VSR	EX	SQ	VSR	EX	SQ
Easy (30)	93%	80%	4.53	100%	97%	5.00
Medium (40)	78%	35%	3.62	92%	43%	4.20
Hard (30)	93%	30%	3.67	87%	47%	3.87
Overall (100)	87%	47%	3.91	93%	60%	4.34

residual V2 failures (IDs 65, 67, 69) reflect incomplete dataset coverage of the LIMIT fix rather than a system defect.

Hard tier. V1 SQ of 3.67 was dominated by LIMIT 10 truncating window-function outputs (e.g., “top 3 crime types per community area” requires 231 rows across 77 areas). Removing the instruction raised V2 Hard EX from 7% to 17% and SQ to 3.87. Four queries (IDs 75, 76, 94, 100) failed due to API rate limiting, reducing Hard VSR to 87%.

3.3 Score Distribution and Failure Analysis

Table 6: SQ score distribution by system

System	Score 5	Score 4	Score 3	Score 2	Score 1
Baseline V1	58	11	10	6	15
Improved V2	69	16	3	4	8
<i>Change</i>	+11	+5	−7	−2	−7

V2 shifts mass from the lower tail to the upper tail: score-1 events fell from 15 to 8, score-3 from 10 to 3, and score-5 events rose from 58 to 69.

Table 7: Root-cause breakdown of low-scoring responses ($SQ \leq 2$)

Failure mode	V1 count	V2 count
Relevance-checker false rejection	10	3
LIMIT 10 result truncation ($SQ=2$)	4	0
API rate-limit failure (empty output)	0	4
Wrong aggregation / interpretation	4	4
Wrong classification logic	1	1
Completely incorrect result	2	1
Total ($SQ \leq 2$)	21	12

The dominant V1 failure mode — relevance-checker false rejections — was reduced by 70% in V2 (10 to 3). The LIMIT 10 truncation failures were fully eliminated for correctly re-run queries. The four API rate-limit failures in V2 are infrastructure issues unrelated to SQL generation logic.

4 Discussion

4.1 Technical Interpretation and Comparative Analysis

The primary V2 improvement driver is removal of the LIMIT 10 system-prompt instruction, which in V1 caused the LLM to truncate all query results regardless of intent — most severely in the Hard tier

where window-function outputs can span hundreds of rows. Removing it raised Hard EX from 7% to 17%. The improved relevance checker adds domain-specific terms (“**breakdown**”, “**distribution**”, “**statistics**”) and action verbs (“**calculate**”, “**identify**”), accounting for most of the Medium VSR improvement (78%→92%). The synthesiser consistently produces high-quality output when SQL is correct ($SQ \geq 4$ in 85/100 V2 rows), confirming SQL generation quality — not language generation — is the primary bottleneck.

V2 exceeded the 4.0 SQ target by 0.34 points; V1 missed by 0.09. The SQ gap is largest at the Easy tier (+0.47), driven by complete elimination of LIMIT truncation, with Medium (+0.58) and Hard (+0.20) gains consistent with the same hypothesis. Strict JSON-match EX (V1 12%, V2 19%) understates functional correctness because the agent frequently uses different column aliases, sort order, or numeric precision than the reference SQL. The hybrid relational equivalence metric — column-name-agnostic value-set comparison escalating to an LLM judge for borderline cases — raises EX to 47% (V1) and 60% (V2), providing a substantially more representative measure of functional correctness.

4.2 Error Analysis

Four persistent failure patterns remain in V2. (1) **Residual relevance rejections (3 queries)**: questions phrased as “which blocks had...”, “what is the distribution of...”, and “what is the median...” lack explicit action verbs from the approved list, causing the fallback branch to reject them incorrectly. (2) **Aggregation scope mismatch (ID 81)**: “average days between consecutive crimes on the same block” was interpreted as a global average (41.3 days) rather than a per-block average; both interpretations are linguistically valid, but the LLM chose the simpler. (3) **Temporal ambiguity (ID 90)**: “the single calendar day with the highest crime count across all years” admits two valid interpretations — highest day-of-year aggregate vs. highest single calendar date — producing different answers (January 1: 5,394 total vs. May 31 2020: 1,901). (4) **Derived-metric disagreement (IDs 87, 90)**: unstated domain conventions such as “night hours” require the LLM to infer cutoffs (e.g., 18:00–06:00 vs. 20:00–05:00), producing numerically different results that are neither obviously correct nor obviously wrong.

4.3 Limitations

- **Rate-limit ceiling**: The `openai/gpt-oss-120b` model has an 8,000 TPM free-tier limit, causing 4 Hard-tier failures and depressing Hard VSR from the expected 93% to 87%. A paid tier or higher-TPM model would eliminate these failures.
- **Strict EX understates correctness**: Exact JSON-match penalises correct answers differing only in alias, sort order, or precision. The hybrid equivalence metric raises V2 EX from 19% to 60%, confirming strict matching systematically understates functional accuracy.
- **Token budget pressure**: The 22-column schema prompt can exceed the 8,000 TPM budget for complex multi-CTE queries (ID 94: 10,238 tokens). Schema compression or selective column injection would alleviate this.
- **Static benchmark**: The 100-question dataset was built before system development; boundary-vocabulary questions (IDs 49, 66, 68) expose a coverage gap requiring continued manual curation or a learned relevance classifier.

5 Future Work

The limitations above share a common root cause: the agent has no memory of prior queries, domain conventions, or successful SQL patterns. Retrieval-Augmented Generation (RAG) [18] backed by a vector database directly addresses each gap.

5.1 Semantic Relevance Checking via Embedding Similarity

Replacing the hand-crafted keyword filter with a vector-database corpus of annotated in-domain questions (embedded via `text-embedding-3-small`) enables cosine-similarity-based relevance scoring

against a learned threshold, eliminating fragile surface-form pattern matching. This directly targets the false rejections responsible for 10 of 21 V1 failures and 3 of 12 V2 failures, and generalizes naturally to paraphrases and domain synonyms the current regex cannot handle.

5.2 Schema-Aware Column Retrieval

The agent currently injects all 22 columns into every prompt (400–600 tokens regardless of query complexity). A vector database of per-column descriptions enables selective injection of the top- k most semantically relevant columns, reducing prompt size by 70–80% for simple queries and freeing token budget for complex multi-CTE queries that currently exceed the 8,000 TPM ceiling (e.g., ID 94 at 10,238 tokens).

5.3 Few-Shot SQL Example Retrieval

A vector store of verified (*question*, *SQL*) pairs enables dynamic few-shot example retrieval at query time. This is especially beneficial for Hard-tier patterns — window functions, rolling averages, multi-CTE pipelines — where zero-shot LLM priors are weakest. Prior work shows that retrieving even three structurally similar examples raises execution accuracy by 10–20 percentage points on standard benchmarks [19].

5.4 Domain Knowledge Store and Evaluation Metric Refinement

Persistent failures from undefined domain conventions (e.g., “night hours”, “calendar day”) can be resolved by a lightweight vector store of factual statements (e.g., “Night hours are defined as 22:00–05:59 in Chicago Police reporting”) injected as context before SQL generation. Concurrently, the hybrid relational equivalence metric should be extended with timestamp normalization and partial-match scoring; BERTScore [20] offers a more robust complement by evaluating faithfulness at the embedding level, better capturing paraphrase-grounded correctness.

References

- [1] B. Qin, B. Hui, L. Wang, B. Li, J. Li, R. Geng, R. Gao, J. Sun, L. Si, F. Huang, and Y. Li, “A survey on text-to-sql parsing: Concepts, methods, and future directions,” *arXiv preprint arXiv:2208.13629*, 2022.
- [2] X. Liu, L. Shi, B. Hui, J. Li, Y. Li, Z. Geng, R. Chen, B. Li, B. Qin, and F. Huang, “A survey of text-to-sql in the era of llms: Where are we, and where are we going?” *arXiv preprint arXiv:2408.05109*, 2024.
- [3] L. Shi, M. Gao, L. Bin, J. Bai, and Z. Ma, “A survey on employing large language models for text-to-sql tasks,” *arXiv preprint arXiv:2407.15186*, 2024.
- [4] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman *et al.*, “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.
- [5] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, “Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.
- [6] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Li, M. Li, and J. Zhou, “Text-to-sql empowered by large language models: A benchmark evaluation,” *arXiv preprint arXiv:2308.15363*, 2024.
- [7] M. Pourreza and D. Raffei, “Din-sql: Decomposed in-context learning of text-to-sql with self-correction,” *arXiv preprint arXiv:2304.11015*, 2023.

- [8] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [9] OpenAI, “GPT-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [10] Defog.ai, “Sqlcoder: State-of-the-art llm for python and sql,” <https://github.com/defog-ai/sqlcoder>, 2023.
- [11] H. Wang, Y. Li, X. Sun, D. Gao, B. Li, M. Li, and J. Zhou, “Mac-sql: A multi-agent collaborative framework for text-to-sql,” *arXiv preprint arXiv:2312.11242*, 2023.
- [12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [13] LangChain Inc., “Langgraph: Building stateful, multi-actor applications with llms,” <https://python.langchain.com/docs/langgraph>, 2024.
- [14] J. Li, B. Hui, G. Qu, B. Li, J. Yang, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, X. Ma, K. Li, W. Liu, W. Jiao, C. Zhou, P. Zhe, and K. Sun, “Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls,” *arXiv preprint arXiv:2305.03111*, 2023.
- [15] G. Katsogiannis-Meimarakis and G. Koutrika, “A survey on deep learning approaches for text-to-sql,” *The VLDB Journal*, vol. 32, no. 4, pp. 905–936, 2023.
- [16] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.
- [17] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *arXiv preprint arXiv:1702.08734*, 2017.
- [18] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [19] C. Guo, Z. Tian, J. Tang, P. Wang, K. Wen, K. Yang, and T. Wang, “Prompting gpt-3.5 for text-to-sql with de-semanticization and skeleton retrieval,” *arXiv preprint arXiv:2304.13301*, 2023.
- [20] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” *arXiv preprint arXiv:1904.09675*, 2020.